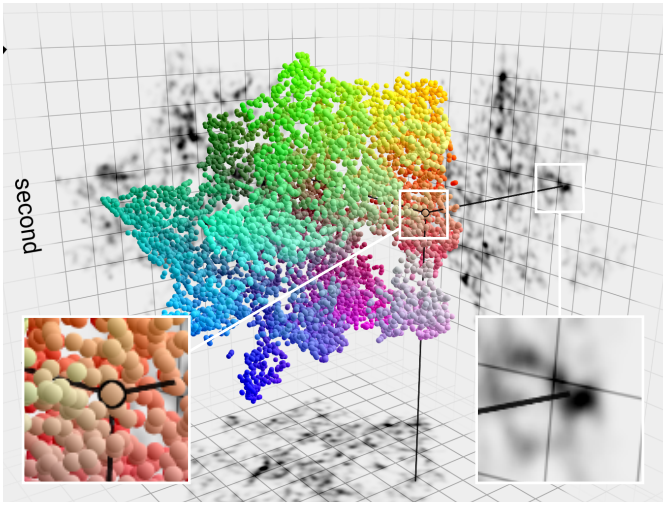
Hardware-accelerated Rendering of Web-based 3D Scatter Plots with Projected Density Fields and Embedded Controls

Lukas Wagner
lukas.wagner@hpi.uni-potsdam.de
Hasso Plattner Institute,
University of Potsdam
Germany

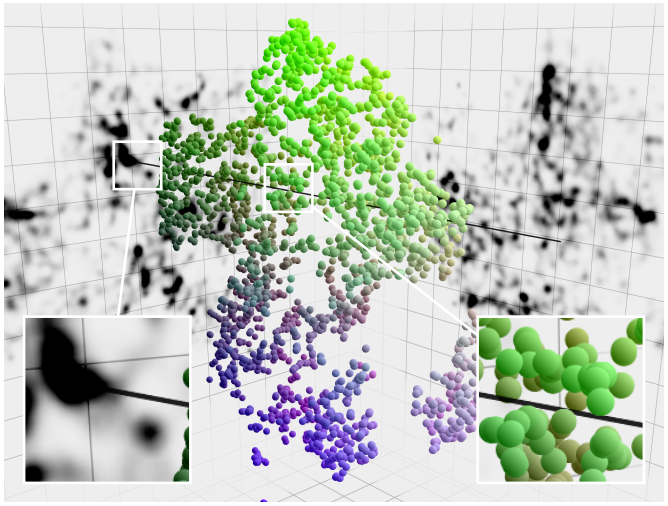
Daniel Limberger
daniel.limberger@hpi.uni-potsdam.de
Hasso Plattner Institute,
University of Potsdam
Germany

Willy Scheibel
willy.scheibel@hpi.uni-potsdam.de
Hasso Plattner Institute,
University of Potsdam
Germany

Jürgen Döllner
juergen.doellner@hpi.uni-potsdam.de
Hasso Plattner Institute,
University of Potsdam
Germany



(a) Example for the interactive exploration of a single data point. The perpendicular lines provide a reference to all three 2D plots.



(b) Example for the exploration of a dense area. The line is perpendicular to the density map, referencing points within the 3D plot.

Figure 1: Two examples of datasets, explored using guides that link the 3D scatterplot to orthographic density maps.

ABSTRACT

3D scatter plots depicting massive data suffer from occlusion, which makes it difficult to get an overview and perceive structure. This paper presents a technique that facilitates the comprehension of heavily occluded 3D scatter plots. Data points are projected to axial planes, creating x-ray-like 2D views that support the user in analyzing the data's density and layout. We showcase our open-source web application with a hardware-accelerated rendering component written in WebGL. It allows for interactive interaction, filtering, and navigation with datasets up to hundreds of thousands of nodes. The implementation is detailed and discussed with respect to challenges posed by API and performance limitations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Web3D '22, November 2–4, 2022, Evry-Courcouronnes, France
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9914-2/22/11…$15.00
https://doi.org/10.1145/3564533.3564566

CCS CONCEPTS

• **Computing methodologies** → **Rendering**; • **Human-centered computing** → *Graphical user interfaces*; *Web-based interaction*; *Information visualization.*

KEYWORDS

scatter plot, 3D visualization, massive data, web

ACM Reference Format:
Lukas Wagner, Daniel Limberger, Willy Scheibel, and Jürgen Döllner. 2022. Hardware-accelerated Rendering of Web-based 3D Scatter Plots with Projected Density Fields and Embedded Controls. In *The 27th International Conference on 3D Web Technology (Web3D '22), November 2–4, 2022, Evry-Courcouronnes, France.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3564533.3564566

1 INTRODUCTION

Scatter plots are commonly used to visualize and analyze relational as well as multivariate data sets. Due to their ease of use and flexibility over data characteristics, they find use in countless domains and variations [Sarikaya and Gleicher 2018; Sedlmair et al. 2013]. The assignment of three scalar variables to the three main axes enables visual exploration of data in 3D. However, as the number of data elements increases, 3D scatter plots increasingly suffer from occlusion. To reduce its negative impact on exploration, the plot can be augmented by a scatter plot matrix.

This paper extends the embedding of a scatter plot matrix into a 3D scatter plot, facilitating the exploration of the density and structure of the depicted data. By providing the user with density plots representing x-ray-like perspectives within the 3D scatter plot, areas of interest become more discernible. We provide interaction techniques that allow filtering out data points associated with a cluster. Our web-based viewer uses WebGL 2 and allows for interactive analysis of multi-million point data sets. It is free to use, hosted on varg.dev[1], and its source code is published on Github[2]. We complement this by discussing relevant API restrictions, storage limitations, and overall computational performance.

2 RELATED WORK

Occlusion and overplotting are well-known issues for 3D and 2D scatterplots, respectively, and continue to be discussed in the field of (interactive) visualization [Elmqvist and Tsigas 2008]. The majority of data elements within massive data sets displayed by a 3D scatter plot will be hidden behind or even surrounded on all sides by other data elements. In the following, we provide a brief overview of common mitigation strategies such as highlighting planes, illustrative shadows, aggregation, and clustering.

One basic approach is to use 3D cutting planes, which hide all objects that fall on one side. Placing the cutting planes requires knowledge about any data of interest being occluded by other data. A variant of this approach are highlight planes, which reduce the scene elements to those close to a user-defined plane [Prouzeau et al. 2019]. Illustrative shadows address occlusion by embedding an additional view of the observed object into the scene [Ritter et al. 2003]. This approach projects the model onto a 2D plane. Objects of interest are highlighted using non-photorealistic rendering.

Another approach for reducing occlusion issues is the reduction of scene elements. For exploring large data sets, clustering allows gaining a overview over the data set by reducing the number of visible elements. Rendering clusters as solid meshes instead of scatter plots "makes [...] use of the innate 3D scene understanding capabilities of the human visual system" [Wang and Mueller 2019]. Splatterplots is an aggregation approach dedicated to scatter plots [Mayorga and Gleicher 2013]. Instead of clustering, a less computation-heavy approach is binning. Typically, the data points are grouped into hyperrectangles, however, other patterns can be used as well [Heimerl et al. 2018]. Alternatively to single-level aggregation, it's also possible to use hierarchical aggregations [Elmqvist and Fekete 2010]. This has the advantage of providing multiple layers of abstraction. Instead of reducing visible data points using aggregation, the points can also be filtered by some metric. Building on Splatterplots, it is possible to filter the data points to represent the splats [Chen et al. 2014].

Another challenge for 3D scatter plots is spatial perception. There are different approaches to improving readability. A natural first approach is the addition of shading. One possible option is cushion shading, which utilizes the fragment position to efficiently approximate realistic shading [van Wijk and van de Wetering 1999]. Improving over this, ambient occlusion provides visual hints about spatial relations. For scatter plots, specific point-based ambient occlusion algorithms exist [Eichelbaum et al. 2013]. Furthermore, the restrictions of a 2D screen can be circumvented by using virtual reality headset or similar 3D technologies. All of these, however, do not solve the occlusion issues.

A possible solution for occlusion in 3D scatter plots are scatter plots matrices. Scatter plot matrices can also be used to navigate high-dimensional data sets [Sanftmann and Weiskopf 2012]. The typical 2D matrix approach, however, does not provide much in the way of discovering the relationships between the graphs. For three dimensions, this can be improved by placing these graphs orthogonal to the three main axes. This adapts the scatter plot matrix, which is purely juxtapositional, into a combination of compositions [Javed and Elmqvist 2012]: The 2D plots are nested into the 3D scene. Visually, however, the 2D plots are either juxtaposed or superimposed with the 3D plot, based on the viewing angle. Such an integration of 2D scatter plots is already supported in some 3D scatter plot viewers. One variant of this are Regression Cubes, especially when examining both a 3D view and the three axis-aligned 2D facets [Chan et al. 2014]. In sufficiently complex scenes, however, the 2D scatter plots may suffer from over-plotting, in turn hindering insights into the 3D visualization.

One approach to solving over-plotting issues is reducing the opacity of the rendered points, e.g. by dynamically selecting an opacity based on the local density of points [Matejka et al. 2015]. Apart from opacity, other design parameters, such as the point size, can be automatically adjusted as well [Micallef et al. 2017]. It is also possible to provide the user with more detailed views of high-density areas [Eisemann et al. 2014]. In order to improve readability of 2D scatter plots, it's also possible to smooth out the plotted data using statistical metrics [Deutsch 1996]. This results in minor details being dropped, letting the user focus on the data set's macro-structure.

3 VISUALIZATION DESIGN

The data set displayed by a 3D scatter plot is contained in an axis-aligned bounding box. Typically, the six bounding planes provide the user with a grid for orientation. They can be used as billboards that visualize additional information. We use an orthographic projection to plot the data on these bounding planes. As known from architectural drawings, the resulting 2D scatter plots are comparable to the ground plan, front elevation, and side elevation. These views represent the different panes of a scatter plot matrix.

In order to eliminate the over-plotting we propose creating density maps instead of simple 2D scatter plots. Thereby, more than one data point can affect the resulting scatter plot at any rasterized location. When plotting the data points, we render a gradient,

[1] demo.varg.dev/vidi/web3d-2022
[2] github.com/lukaswagner/vidi

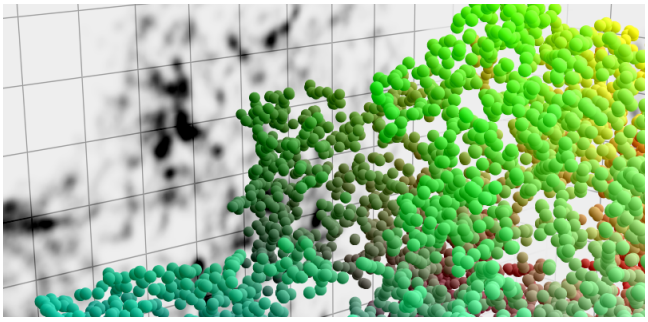
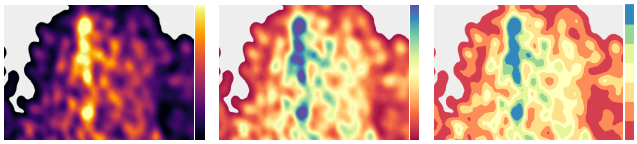


Figure 2: Additive blending of graphical elements into a density map reveals data clusters.



(a) Inferno **(b) Spectral, 64 steps** **(c) Spectral, 7 steps**

Figure 3: The same density map rendered with different color schemes (3a and 3b) and color resolutions (3b and 3c).

starting with the maximum density at the point's center and falling off with increasing distance. By additively blending all points, we obtain an image where each pixel contains information about how many points are close to it. The density maps are then be projected onto the 3D scatter plot's grids, as shown in Figure 2. This allows the user to spot and locate clusters of data points which may be occluded and concealed otherwise.

We map the intensity to a color scale to improve its readability. Figure 3 gives an overview of example color schemes. While different color schemes do not change the actual data being displayed, they can help by increasing the output resolution and connecting the data to more accustomed concepts. Alternatively, we can reduce the resolution of the color scale which results in more discernible color bands that denote areas of equal density (contour intervals).

In order to increase the scatter plot's interactive capabilities, we use graphical control elements integrated within the 3D scene. As the browser only provides interaction with HTML elements, this requires implementation of techniques such as picking (section 4). The user might want to filter and locate data points belonging to certain areas within the density maps. This is supported by hiding any data points occluding the area of interest. For this use case, we provide the user with sliders controlling axis aligned cutting planes. These controls are placed co-planar to the grids and density maps. This allows using the grid lines as reference for placing the control handles based on the density maps.

Another challenge is deducting a 3D point's position on the 2D density maps. When hovering the cursor over a data point, we provide the user with reference lines to the respective position on all three planes (Figure 4). These reference lines are axis-aligned and thus perpendicular on the planes. Inversely, we also provide the user with reference lines from a 2D map into the 3D scatter plot. By clicking on any of the density maps, the user can place

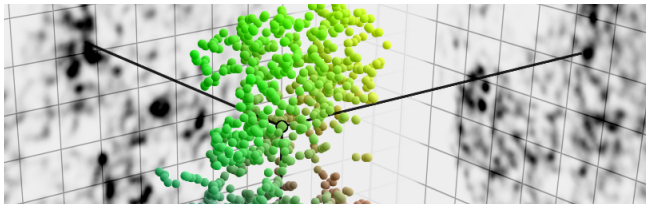


Figure 4: Reference lines to all three planes provide easy evaluation of a point's position on the density maps. Here, the selected data point shares its position with many others on the right and left density maps.

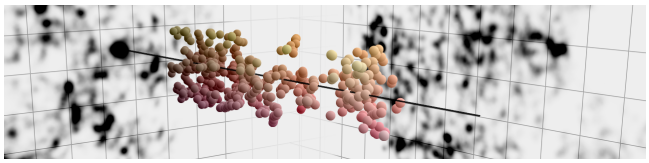


Figure 5: By placing a reference line onto any of the density maps, the points composing a cluster can be deducted.

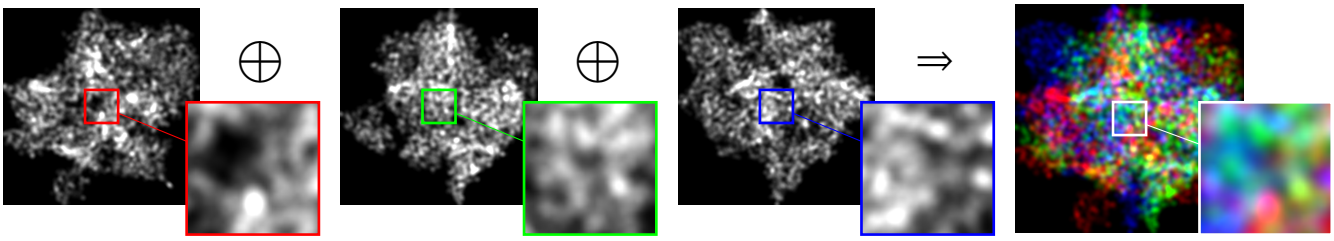
such a line. This reference line once again is placed perpendicular on the respective plane (Figure 5). Together, these reference line interaction techniques facilitate cross-references between the 3D scatter plot and the 2D density maps.

4 IMPLEMENTATION

Our implementation uses WebGL 2 to provide a web-based data set introspection tool [Wagner et al. 2020]. Depending on the GPU, screen resolution and rendering settings, the maximum number of data points for interactive rendering ranges from 1 million to above 50 million.

Density Maps. The density maps are generated by additional render passes, using axis aligned orthographic projections. Thus, each vertex has to be rendered four times instead of once. However, as the density maps do not change once the data set is loaded, the additional render passes can be run once instead of on every frame. For encoding the density maps, we chose a 16-bit floating point RGB texture, with each channel storing one of the three maps (Figure 6). In order to count the data points overlapping at any position in the resulting density map, we utilize additive blending, which requires configuring the blending function to `gl.blendFunc(gl.ONE, gl.ONE)`. This results in all density values being added up without being influenced by alpha blending.

Density Gradients. To create density gradients, we encode the distance to the point's center instead of drawing each data point as a uniform area—similar to how metaballs are created. Instead of an inverse falloff, however, we chose a linear falloff. This results in a smoother density map, having close points melt together earlier. An inverse falloff function would result in more abrupt density changes, with points staying separated longer but merging quicker once their distance falls under a threshold.



(a) Red channel contains front view (x points right, y points up) **(b) Green channel contains side view (z points left, y points up)** **(c) Blue channel contains top view (x points right, z points down)** **(d) Combined texture containing all three density maps.**

Figure 6: Packed density maps, each color channel storing one map. The values are scaled down to 5 %, however, some areas still appear blown out, as the number of data points at that position exceeds the [0, 1] range typically expected in an RGB texture.

Density Maps Scaling. In order to provide a useful default scaling for the density maps, the density should be normalized. This requires the maximum density to be determined. A trivial approach would be to read back all texels from the GPU's memory and calculate the maximum using the CPU. Instead, we render the texture to a lower resolution texture using a screen aligned quad, with each output texel calculated as the maximum of an $n \times n$ kernel, n being the ratio between the textures' resolutions. For example, a 512×512 texture can be reduced to a single value using three passes with an 8×8 kernel ($512 \rightarrow 64 \rightarrow 8 \rightarrow 1$). The value can then be read back. This is a common approach to GPU-accelerated reduction [Buck and Purcell 2004; Sarikaya and Gleicher 2015]. Dividing by the maximum density gives a sensible default scaling for the density values, each ranging between 0 and 1. The scaling can also be adjusted using strength and gamma adjustments, as known from image manipulation.

Color Schemes. The different color schemes can be realized by providing an $n \times 1$-texture to the fragment shader. This texture is filled with values on the CPU side. Resizing the texture and disabling linear filtering allows for lower color resolution (Figure 3c).

Interaction. Point selection is implemented using an ID buffer. IDs are stored as 32-bit unsigned integers, resulting in an maximum supported point count of over 4 billion. If more distinct IDs are required, the IDs could be split into a high and low part, which are stored as separate variables. By reading back the pixel below the mouse cursor, the currently active data point can be determined. For placing reference lines on the density maps, this approach is reused. Reading this data back from the GPU allows discerning where the user clicked, as well as on which of the three planes. As the IDs are stored as an 32-bit integer, this data is packed into the same amount of bits. One byte is used to store the grid ID. The remaining 3 bytes store the x and y position on the grid, each encoded as a 12-bit integer. The precision of 4096 different positions per axis is enough for typical rendering scenarios.

Reference Lines. For rendering the reference lines, the points' vertex attribute array is rebound with an offset based on the selected ID. This allows reusing the data already present in GPU memory. Setting a coordinate's value to the respective grid's position results in a perpendicular line, which is repeated for all three grids.

5 EVALUATION

This evaluation is split into two parts. First, we assess whether 16-bit floating point textures provide enough precision for our use case. Afterwards, we discuss this technique's performance impact.

Density Map Precision. When describing the density map generation, we noted that it is stored as an 16-bit floating point texture. In comparison with a 32-bit integer texture, this means a significant drop in precision. Even when using a 32-bit floating point texture, only 24 bits are used to store the mantissa (counting the hidden bit)[3]. This affects how many points can overlap before any data loss occurs. The required precision is dependent on both the point size and the falloff function, as well as the target number of decimal places. Storing 32-bit fixed point numbers with a precision of 0.01 would allow over 40 million points to overlap, while allowing each point to contribute a full density of 1 ($n = 2^{32} * 0.01$). However, WebGL 2 does not support blending of any type for integer textures. This means we can't utilize additive blending to aggregate density values in an integer texture and therefore must build on floating point textures. A 32-bit floating point number however only allows for around 168 thousand overlapping points before data loss occurs at the second decimal ($n = 2^{24} * 0.01$).

32-bit textures introduce an additional issue, as WebGL 2 does not support bi-linear interpolation[4]. This results in pixelated density maps. It is possible to manually implement the linear filtering inside the fragment shader, however this results in a performance impact. In order to utilize hardware-accelerated interpolation, the bit-depth must be reduced to 16. This, in turn, reduces the mantissa to 11 bits (again including the hidden bit), resulting in a significantly lower number of overlapping points of 20 ($n = 2^{11} * 0.01$). While heavily reduced, this precision could still be considered acceptable based on the number of data points and point size. Assuming a normal distribution, around 0.04 % of points affect the center pixel[5]. A resolution of 11 bit thus is sufficient for data sets of up to 50 thousand points ($n = 2^{11} * 0.01/(0.02^2)$). Relaxing the precision requirement to 1 digit increases this to 500 thousand points.

[3] www.khronos.org/opengl/wiki/Small_Float_Formats
[4] www.khronos.org/registry/OpenGL-Refpages/es3.0/html/glTexImage2D.xhtml
[5] Resolution 512 px, 5 px point size. Using a mean of 255,5 and a standard derivation of 100, approximately 2 % of points fall into the [253, 258] range, affecting the center pixel. Extending this to 2D results in only $0.02 * 0.02$, or 0.04 % of points. See onlinestatbook.com/2/calculators/normal_dist.html

To support more points, the density calculation could use multiple frame buffers and distribute the points heuristically, e.g., based on the point ID. These could be added up in a post processing step.

In order to implement the distribution of points as minimally invasive as possible, one possible approach would be to extend the frame buffer by additional attachments. Based on the the chosen heuristic, the fragment shader writes only to one of them. This requires no adjustments to the underlying rendering process. This approach, however, is restricted by the maximum possible number of color attachments. The OpenGL ES 3 specification (which WebGL 2 is based on) requires `MAX_COLOR_ATTACHMENTS` to be at least 4, with modern devices often supporting more attachments.

Performance. The density map calculation takes less time than rendering the 3D scene. This results from a reduced resolution (e.g., 256×256 instead of 1920×1080, resulting in a fragment reduction by factor 31.6), as well as the simpler fragment shader. On top of this, the density maps can be cached and reused as long as the underlying data is unchanged. In practice, this means that the density maps' performance impact is limited to texture lookups when drawing the grids. Thus, we confirmed earlier performance measurements, meaning that millions of data points can be interactively visualized on mid-range consumer hardware [Wagner et al. 2020].

6 CONCLUSIONS

This paper proposes a novel technique for exploring 3D scatter plots suffering from occlusion. We extended the existing approach of mapping 2D scatter plots onto axis-aligned grids with enhancements for resolving 2D over-plotting issues. This results in density maps, condensing individual points into an overview of the data set's structure. We combined this with interaction techniques for filtering and referencing data points based on the density maps. This approach facilitates the discovery and exploration of data sets.

The proposed techniques provide the user with an overview of the data set. This approach works alongside the 3D scatter plot but does not actively affect it. A more active interaction could further improve data set comprehension, e.g., by affecting visual components based on density. One possible enhancement is reducing the opacity of data points in low-density areas, resulting in a scatter plot focused on high-density clusters. Instead of displaying the density maps, the bounding planes could also be used to display additional data, such as aggregated point data. A comparison of different density falloff functions could also be worthwhile. The current implementation uses a linear falloff based on the intent of counting overlapping data points. Another option, inspired by metaballs, is the inverse squared euclidean distance.

ACKNOWLEDGMENTS

This work was funded by the German Federal Ministry of Education and Research (BMBF, KMUi) within the project "Knowhow-Analyzer". This work is part of the "Software-DNA" project, which was funded by the European Regional Development Fund (ERDF – or EFRE in German) and the State of Brandenburg (ILB). This Project is supported by the Federal Ministry for Economic Affairs and Climate Action (BMWK, ZIM) on the basis of a decision by the German Bundestag.

REFERENCES

Ian Buck and Tim Purcell. 2004. A Toolkit for Computation on GPUs. In *GPU Gems*. Addison Wesley Professional, Chapter 37.

Yu-Hsuan Chan, Carlos D. Correa, and Kwan-Liu Ma. 2014. Regression Cube: A Technique for Multidimensional Visual Exploration and Interactive Pattern Finding. *Transactions Interactive Intelligent Systems* 4, 1 (2014), 7:1–32 pages. https://doi.org/10.1145/2590349

Haidong Chen, Wei Chen, Honghui Mei, Zhiqi Liu, Kun Zhou, Weifeng Chen, Wentao Gu, and Kwan-Liu Ma. 2014. Visual Abstraction and Exploration of Multi-class Scatterplots. *Transactions on Visualization and Computer Graphics* 20, 12 (2014), 1683–1692. https://doi.org/10.1109/TVCG.2014.2346594

Clayton V. Deutsch. 1996. Constrained Smoothing of Histograms and Scatterplots with Simulated Annealing. *Technometrics* 38, 3 (1996), 266–274. https://doi.org/10.2307/1270610

Sebastian Eichelbaum, Gerik Scheuermann, and Mario Hlawitschka. 2013. PointAO – Improved Ambient Occlusion for Point-based Visualization. In *Proc. EuroVis – Short Papers*. EG, 13–17. https://doi.org/10.2312/PE.EuroVisShort.EuroVisShort2013.013-017

Martin Eisemann, Georgia Albuquerque, and Marcus Magnor. 2014. A Nested Hierarchy of Localized Scatterplots. In *Proc. SIBGRAPI Conference on Graphics, Patterns and Images*. IEEE, 80–86. https://doi.org/10.1109/SIBGRAPI.2014.14

Niklas Elmqvist and Jean-Daniel Fekete. 2010. Hierarchical Aggregation for Information Visualization: Overview, Techniques, and Design Guidelines. *Transactions on Visualization and Computer Graphics* 16, 3 (2010), 439–454. https://doi.org/10.1109/TVCG.2009.84

Niklas Elmqvist and Philippas Tsigas. 2008. A Taxonomy of 3D Occlusion Management for Visualization. *Transactions on Visualization and Computer Graphics* 14, 5 (2008), 1095–1109. https://doi.org/10.1109/TVCG.2008.59

Florian Heimerl, Chih-Ching Chang, Alper Sarikaya, and Michael Gleicher. 2018. Visual Designs for Binned Aggregation of Multi-Class Scatterplots. *arXiv CoRR* cs.HC (2018), 14 pages. https://doi.org/10.48550/ARXIV.1810.02445

Waqas Javed and Niklas Elmqvist. 2012. Exploring the design space of composite visualization. In *Proc. Pacific Visualization Symposium (PacificVis '12)*. IEEE, 1–8. https://doi.org/10.1109/PacificVis.2012.6183556

Justin Matejka, Fraser Anderson, and George Fitzmaurice. 2015. Dynamic Opacity Optimization for Scatter Plots. In *Proc. 33rd Conference on Human Factors in Computing Systems (CHI '15)*. ACM, 2707–2710. https://doi.org/10.1145/2702123.2702585

Adrian Mayorga and Michael Gleicher. 2013. Splatterplots: Overcoming Overdraw in Scatter Plots. *Transactions on Visualization and Computer Graphics* 19 (2013), 1526–1538. https://doi.org/10.1109/TVCG.2013.65

Luana Micallef, Gregorio Palmas, Antti Oulasvirta, and Tino Weinkauf. 2017. Towards Perceptual Optimization of the Visual Design of Scatterplots. *Transactions on Visualization and Computer Graphics* 23, 6 (2017), 1588–1599. https://doi.org/10.1109/TVCG.2017.2674978

Arnaud Prouzeau, Maxime Cordeil, Clement Robin, Barrett Ens, Bruce H. Thomas, and Tim Dwyer. 2019. Scaptics and Highlight-Planes: Immersive Interaction Techniques for Finding Occluded Features in 3D Scatterplots. In *Proc. CHI Conference on Human Factors in Computing Systems (CHI '19)*. ACM, 1–12. https://doi.org/10.1145/3290605.3300555

Felix Ritter, Henry Sonnet, Knut Hartmann, and Thomas Strothotte. 2003. Illustrative Shadows: Integrating 3D and 2D Information Displays. In *Proc. 8th International Conference on Intelligent User Interfaces (IUI '03)*. ACM, 166–173. https://doi.org/10.1145/604045.604072

Harald Sanftmann and Daniel Weiskopf. 2012. 3D Scatterplot Navigation. *Transactions on Visualization and Computer Graphics* 18, 11 (2012), 1969–1978. https://doi.org/10.1109/TVCG.2012.35

Alper Sarikaya and Michael Gleicher. 2015. Using WebGL as an Interactive Visualization Medium: Our Experience Developing SplatterJs. In *Proc. Data Systems for Interactive Analysis Workshop (DSIA '15)*. 4 pages.

Alper Sarikaya and Michael Gleicher. 2018. Scatterplots: Tasks, Data, and Designs. *Transactions on Visualization and Computer Graphics* 24, 1 (2018), 402–412. https://doi.org/10.1109/TVCG.2017.2744184

Michael Sedlmair, Tamara Munzner, and Melanie Tory. 2013. Empirical Guidance on Scatterplot and Dimension Reduction Technique Choices. *Transactions on Visualization and Computer Graphics* 19, 12 (2013), 2634–2643. https://doi.org/10.1109/TVCG.2013.153

Jarke J. van Wijk and Huub van de Wetering. 1999. Cushion treemaps: Visualization of hierarchical information. In *Proc. Symposium on Information Visualization (InfoVis '99)*. IEEE, 73–78. https://doi.org/10.1109/INFVIS.1999.801860

Lukas Wagner, Daniel Limberger, Willy Scheibel, Matthias Trapp, and Jürgen Döllner. 2020. A Framework for Interactive Exploration of Clusters in Massive Data Using 3D Scatter Plots and WebGL. In *Proc. 25th International Conference on 3D Web Technology (Web3D '20)*. ACM, 31:1–2. https://doi.org/10.1145/3424616.3424730

Bing Wang and Klaus Mueller. 2019. Subspace Shapes: Enhancing High-Dimensional Subspace Structures via Ambient Occlusion Shading. *arXiv CoRR* cs.HC (2019), 8 pages. https://doi.org/10.48550/ARXIV.1911.07447